Apex Institute of Technology

Computer Science & Engineering

Experiment 10

Name: Trimann Kaur **UID: 24BAI70511**

Branch: B.E. CSE (AIML) **Section: 24AIT_KRG-G1**

Semester: 4 **Date of Performance: 24.04.2026**

Subject Name: Database Management System **Subject Code: 24CSH-298**

AIM: To design a trigger that automatically implements the functionality of a primary key, ensuring unique identification of records without manual intervention.

OBJECTIVE:

- To create a database trigger that automatically enforces primary key constraints or generates unique key values, replicating the functionality of a stored procedure.

SOFTWARE REQUIREMENTS:

- Database Management System:
  - PostgreSQL Database
- Database Administration Tool:
  - pgAdmin

PRACTICAL/EXPERIMENT STEPS:

1. Identify the table requiring automated primary key enforcement.
2. Design a trigger that activates before insert operations.
3. Ensure that every new record receives a unique primary key automatically.
4. Validate the trigger by inserting multiple records and verifying unique keys.

PROCEDURE:

1. The PostgreSQL environment (pgAdmin / psql) was opened and the required database was selected.
2. The employees table was created using the CREATE TABLE command with emp_id as the primary key.

3. A sequence emp_seq was created using the CREATE SEQUENCE command to generate unique values.
4. A trigger function generate_emp_id() was created to assign emp_id automatically using nextval('emp_seq').
5. A trigger trg_generate_emp_id was defined to execute BEFORE INSERT on the employees table.
6. The trigger was associated with the function to ensure automatic execution during insertion.
7. Sample records were inserted into the table without specifying emp_id.
8. The table was queried using SELECT to verify that unique primary keys were generated automatically.

CODE:

```
CREATE TABLE employees (
   emp_id INT PRIMARY KEY,
   name VARCHAR(100),
   department VARCHAR(50)
);

CREATE SEQUENCE emp_seq
START 1;

CREATE OR REPLACE FUNCTION generate_emp_id()
RETURNS TRIGGER AS $$
BEGIN
   IF NEW.emp_id IS NULL THEN
      NEW.emp_id := nextval('emp_seq');
   END IF;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_generate_emp_id
BEFORE INSERT ON employees
FOR EACH ROW
EXECUTE FUNCTION generate_emp_id();


INSERT INTO employees (name, department)
VALUES ('Alice', 'HR');
```

```
INSERT INTO employees (name, department)
VALUES ('Bob', 'IT');

SELECT * FROM employees;
```

I/O ANALYSIS:

1. Create Table
   The employees table is created with fields emp_id, name, and department, where emp_id is defined as the primary key to ensure unique records.

```
CREATE TABLE

Query returned successfully in 475 msec.
```

2. Create Sequence
   The sequence emp_seq is created successfully to generate unique incremental values for the primary key.

```
CREATE SEQUENCE

Query returned successfully in 144 msec.
```

3. Create Trigger Function
   The trigger function generate_emp_id() is created to automatically assign a unique emp_id using the sequence before insertion.

```
CREATE FUNCTION

Query returned successfully in 158 msec.
```

4. Create Trigger
   The trigger trg_generate_emp_id is defined to execute BEFORE INSERT on the employees table, enabling automatic key generation.

```
CREATE TRIGGER

Query returned successfully in 419 msec.
```

5. Insert Records
   Records are inserted without specifying emp_id, and the trigger automatically assigns unique values to each record.

```
INSERT 0 1

Query returned successfully in 153 msec.
```

6. Verify Output
   The SELECT query confirms that all records have unique emp_id values, ensuring proper trigger execution and data integrity.

	emp_id [PK] integer	name character varying (100)	department character varying (50)
1	1	Alice	HR
2	2	Bob	IT

LEARNING OUTCOMES:

1. Understood the purpose and working of database triggers.
2. Implemented automated primary key functionality using triggers.
3. Ensured data integrity without manual key assignment.